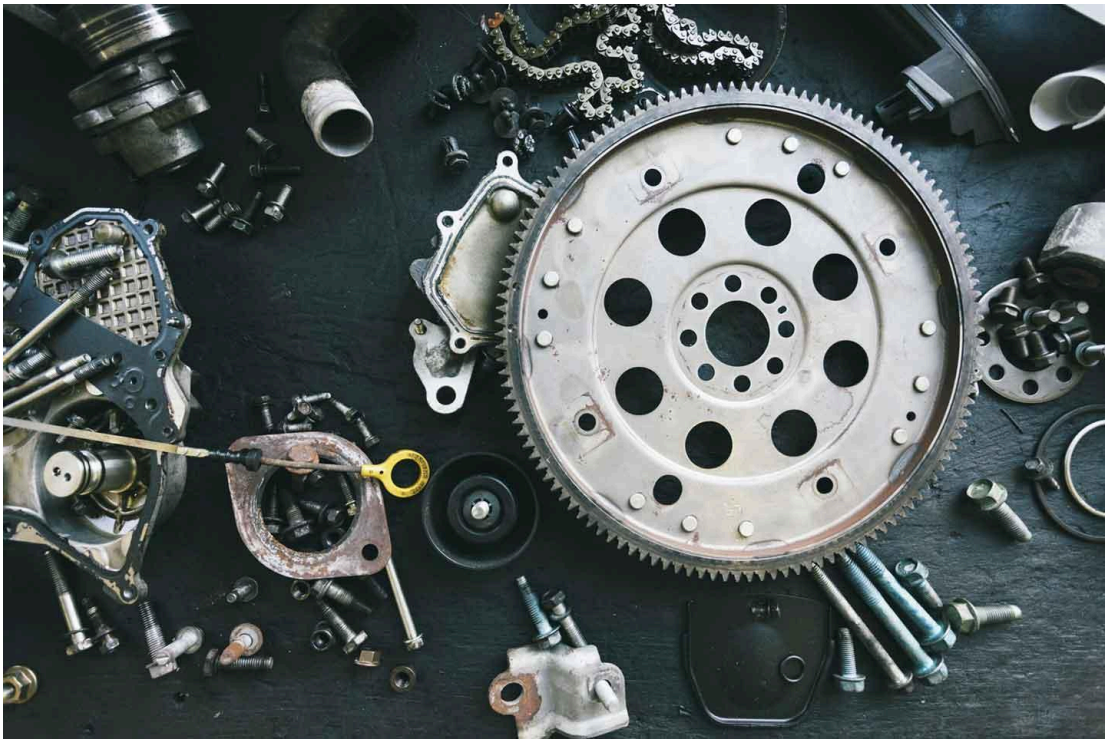
Tema 3

Spring Data JPA y Repositorios

Digitech
FP OFICIAL DIGITAL & TECH



Francisco Belda

ÍNDICE

Teoría. ¿Qué es Spring Data JPA? 3
Paso a Paso: Creación del Repositorio 4
Creación del paquete repository 4
Definición de la interfaz EstudianteRepository 4
Desglose técnico de la interfaz 4
Práctica: Probando el Repositorio con CommandLineRunner 5
Implementación del componente de prueba 5
Ejecución y Comprobación 6
Ejercicios 7
Conceptos Clave Recordados 7
Ejercicios 7

TEORÍA. ¿QUÉ ES SPRING DATA JPA?

Escribir consultas SQL repetitivas para operaciones básicas (como insertar un registro, buscar por ID o actualizar una fila) consume mucho tiempo y es propenso a errores. Spring Data JPA es un subproyecto dentro del paraguas de Spring que **simplifica enormemente la implementación de la capa de acceso a datos**.

Su mayor ventaja es el patrón **Repository**: al definir una interfaz de Java que extienda de JpaRepository, Spring Data **implementa automáticamente todas las operaciones CRUD** comunes en tiempo de ejecución, sin necesidad de que escribas ni una sola línea de SQL o código de implementación.

Paso a Paso: Creación del Repositorio

Creación del paquete repository

Crea un nuevo **paquete** llamado **repository** dentro de tu estructura raíz (ej. **com.educacion.demospring.repository**).

Definición de la interfaz EstudianteRepository

Crea una nueva **interfaz** llamada **EstudianteRepository.java** dentro del paquete recién creado:

```
package com.educacion.demospring.repository;


import com.educacion.demospring.model.Estudiante;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;


@Repository

public interface EstudianteRepository extends JpaRepository<Estudiante, Long> {

    // Hereda métodos CRUD automáticos sin necesidad de implementarlos

    boolean existsByCorreo(String correo);

}
```

Desglose técnico de la interfaz

`@Repository`: Anotación opcional pero recomendada que indica que esta interfaz actúa como un componente de acceso a datos (además traduce excepciones de base de datos a excepciones desacopladas de Spring).

`JpaRepository<Estudiante, Long>`: Es una interfaz genérica que recibe dos parámetros:

La entidad que va a gestionar (**Estudiante**).

El tipo de datos de su clave primaria (**Long**, ya que nuestro ID es de ese tipo).

Al extender de **JpaRepository**, heredamos automáticamente métodos listos para usar como:

save(S entity): Guarda o actualiza un estudiante.

findById(ID id): Busca un estudiante por su clave primaria (devuelve un Optional<Estudiante>).

findAll(): Devuelve una lista con todos los estudiantes registrados.

deleteById(ID id): Elimina un registro por su identificador.

count(): Devuelve el número total de registros.

Práctica: Probando el Repositorio con CommandLineRunner

Para comprobar que nuestro repositorio funciona correctamente y es capaz de comunicarse con la base de datos H2 en memoria, utilizaremos la interfaz CommandLineRunner, la cual permite ejecutar código automáticamente justo después de que Spring Boot haya arrancado el contexto de la aplicación.

Implementación del componente de prueba

```
package com.educacion.demospring.runner;

import com.educacion.demospring.model.Estudiante;

import
com.educacion.demospring.repository.EstudianteRepository;

import org.springframework.boot.CommandLineRunner;

import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;
```

```
@Configuration
public class InicializadorDatos {

   @Bean
           CommandLineRunner    initData(EstudianteRepository
estudianteRepository){
       return args -> {
           // 1. Crear e insertar nuevos estudiantes usando el
método save()
              Estudiante est1 = new Estudiante("Ana Gómez",
"ana.gomez@educacion.es", 20);
             Estudiante est2 = new Estudiante("Carlos Ruiz",
"carlos.ruiz@educacion.es", 22);
           estudianteRepository.save(est1);
           estudianteRepository.save(est2);
               System.out.println(">>> Estudiantes guardados
correctamente en la base de datos.");

               // 2. Consultar todos los estudiantes usando
findAll()
                System.out.println(">>> Listado completo de
estudiantes en la BD:");
                                    for    (Estudiante    e    :
estudianteRepository.findAll()) {
                System.out.println("- ID: "+ e.getId()+ " - "
+ e.getNombre() + " (" + e.getCorreo() + ")");
           }

           // 3. Buscamos un estudiante por Id
           System.out.println("\n=== BUSCAR POR ID ===");
```

```
                                Optional<Estudiante>  e  =
estudianteRepository.findById(1L);

            e.ifPresent(estudiante -> System.out.println("Id:
1. " + estudiante.getNombre()));


            // 4. Actualizamos el estudiante con Id 1

            System.out.println("\n=== ACTUALIZAR ID 1 ===");

            // Paso A: Buscar por ID

                        Estudiante  estudianteEncontrado  =
estudianteRepository.findById(1L).orElseThrow();


            // Paso B: Modificar los datos con los Setters

            estudianteEncontrado.setNombre("Sara Gómez");

estudianteEncontrado.setCorreo("sara.gomez@educacion.es");


            // Paso C: Guardar los cambios (JPA detecta el ID 1
y hace UPDATE)

            estudianteRepository.save(estudianteEncontrado);


                System.out.println("\n===  VERIFICAR  EL  CAMBIO
===");

                        System.out.println("Actualizado:  "  +
estudianteRepository.findById(1L).orElse(null));


            // 5. Contamos los estudiantes

            System.out.println("\n=== CONTAMOS LOS ESTUDIANTES
===");

              System.out.println("El  total  de  estudiantes  es:
"+estudianteRepository.count());


            // 6. Borramos el estudiante con ID 2
```

```
            System.out.println("\n=== BORRAMOS EL ID 2 ===");
            estudianteRepository.deleteById(2L);

            // 6.1 Mostramos como queda la BD
                System.out.println(">>> Listado completo de estudiantes en la BD:");
                for (Estudiante est : estudianteRepository.findAll()) {
                System.out.println("- ID: "+ est.getId()+ " - " + est.getNombre() + " (" + est.getCorreo() + ")");
            }
        };
    }
}
```

Ejecución y Comprobación

Ejecuta tu aplicación con **mvn spring-boot:run** y observa la consola. Verás las sentencias SQL de inserción y consulta generadas por Hibernate, junto con los mensajes impresos por nuestro componente indicando que los datos se han persistido y recuperado con éxito.

Ejercicios

Conceptos Clave Recordados

Spring Data JPA elimina la necesidad de escribir código de acceso a datos repetitivo mediante interfaces que extienden de JpaRepository.

El segundo parámetro genérico de JpaRepository define siempre el tipo de dato de la clave primaria de la entidad.

CommandLineRunner es una herramienta excelente en fases de aprendizaje y pruebas para inicializar datos al arrancar la aplicación.

Ejercicios

1. **Crear el Repositorio para Profesor**: Siguiendo el mismo patrón, crea la interfaz **ProfesorRepository** que extienda de **JpaRepository<Profesor, Long>** para la entidad **Profesor** desarrollada en la sesión anterior.
2. **Ampliación del Inicializador**: Modifica el componente **InicializadorDatos** para que también guarde al menos **tres registros de profesores** utilizando su respectivo repositorio, actualice 1, borre otro y muestre por consola los que queden al arrancar la aplicación.